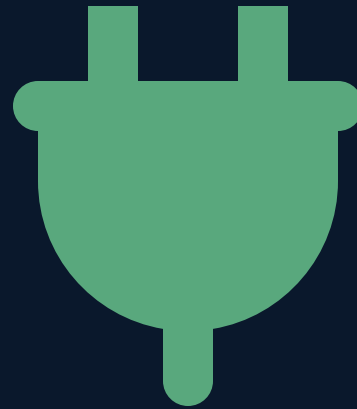


PORTADA

Arquitectura de Software

Semana 5 — Conectores y Protocolos de Garlan y Shaw

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

37 segundos
370 millones de dólares



CASO · FICHA TÉCNICA

Ariane 5, vuelo 501 — 1996

Proyecto	Cohete Ariane 5, Agencia Espacial Europea (ESA) / Arianespace
Fecha	4 de junio de 1996, primer vuelo del Ariane 5
Falla	Autodestrucción activada 37 segundos después del despegue
Pérdida	El cohete y su carga (4 satélites Cluster), estimada en unos 370 millones de dólares
Causa raíz	Un módulo de software reutilizado sin cambios del Ariane 4
Fuente	Informe oficial de la junta de investigación de ESA/CNES (1996)

Uno de los casos más citados en ingeniería de software para ilustrar fallas de integración entre componentes.

CASO · EL COMPONENTE REUTILIZADO

Un componente probado, en un contexto nuevo

El **Sistema de Referencia Inercial (SRI)** calcula la orientación y velocidad del cohete. Había funcionado sin problemas en decenas de vuelos del Ariane 4 — un cohete **más lento** en su fase inicial de ascenso.

El equipo reutilizó ese mismo módulo en el Ariane 5 **sin volver a analizar sus supuestos internos**, confiando en que un componente ya probado seguiría siendo válido en un cohete distinto.

CASO · EL DESAJUSTE TÉCNICO

Un número que no cupo en su casilla

El SRI convertía un valor de **velocidad horizontal**, representado internamente en 64 bits, a un formato de **16 bits** para pasarlo a otro componente. El Ariane 5 acelera mucho más rápido que el Ariane 4 en sus primeros segundos — y ese valor superó lo que el formato de 16 bits podía representar.

- La conversión produjo un **desbordamiento (overflow)** no controlado.
- El software no tenía manejo de excepción para ese caso — se apagó.
- Un segundo SRI de respaldo, idéntico, falló exactamente por la misma razón, al mismo tiempo.

CASO · LA CADENA HASTA EL DESASTRE

De un dato mal formado a la autodestrucción

1. El SRI principal se apaga por el desbordamiento no manejado.
2. El SRI de respaldo, con el mismo software, falla por la misma causa, milisegundos después.
3. El sistema de control de vuelo recibe datos de diagnóstico sin sentido, interpretándolos como datos de vuelo válidos.
4. El cohete ejecuta una maniobra correctiva extrema basada en esos datos falsos.
5. Las fuerzas aerodinámicas resultantes activan la autodestrucción automática.

El código del SRI no tenía ningún defecto de lógica — el problema fue cómo se conectaba con el resto del sistema.

TRANSICIÓN

El problema no era el componente

El SRI, como componente aislado, hacía exactamente lo que siempre había hecho. **Lo que falló fue la relación entre componentes:** qué formato de datos se transmitía, qué suposiciones tenía cada lado sobre el rango de valores posibles, y qué pasaba si esas suposiciones no se cumplían.

A ese "cómo se relacionan los componentes" es a lo que Garlan y Shaw llaman conector — el tema de hoy.

CONCEPTO · REPASO

Componentes, otra vez

Como se vio en la Semana 1 con la definición de Garlan & Shaw, un sistema está hecho de **componentes** (unidades de cómputo con una función, como el SRI) y de algo que los conecta. Hoy nos enfocamos en esa segunda pieza.

CONCEPTO

¿Qué es un conector?

Un **conector** (Garlan & Shaw, 1994) es el elemento arquitectónico que representa **la interacción entre componentes**: transmite datos y control entre ellos, sin ser en sí mismo un componente que resuelve una función de negocio.

Incluye tanto el mecanismo de comunicación como el conjunto de reglas —el protocolo— que rige esa comunicación.

En Ariane 5: el conector entre el SRI y el sistema de control de vuelo, incluyendo el formato de datos acordado (o mal acordado) entre ambos.

CONCEPTO

Taxonomía de conectores

Tipo de conector	Cómo se comunican los componentes
Llamada a procedimiento	Un componente invoca directamente una función de otro y espera su resultado
Invocación implícita (eventos)	Un componente anuncia un evento; otros reaccionan sin que el primero los llame directamente
Flujos de datos (pipes)	La salida de un componente se convierte en la entrada de otro, en secuencia
Acceso a datos compartidos	Varios componentes leen y escriben sobre un mismo repositorio de datos

Estos tipos de conector son la base de los patrones arquitectónicos que se estudian en la Unidad 2 (Tuberías y Filtros, Blackboard, Brokers).

CONCEPTO

El protocolo: las reglas del conector

Todo conector implica un **protocolo**: el conjunto de reglas que especifica cómo debe darse la interacción — orden de los mensajes, formato de los datos, qué hacer ante un error, qué garantías se ofrecen.

- ¿En qué formato viaja el dato? (el punto exacto donde falló Ariane 5)
- ¿Qué pasa si el receptor no responde a tiempo?
- ¿Es una comunicación síncrona (se espera respuesta) o asíncrona?

HTTP, visto en la Semana 1, es un protocolo concreto para un conector de tipo llamada a procedimiento remota.



CONTRAEJEMPLO

Mito: "un conector es solo una función que llama a otra"

La llamada a procedimiento es el conector más intuitivo, pero **reducir "conector" a eso deja fuera la mayoría de los sistemas reales**: colas de mensajes, buses de eventos, bases de datos compartidas, streams de datos entre microservicios.

El caso Ariane 5 muestra, además, que el conector no es solo "cómo se llaman" los componentes — es también qué formato de datos se transmite y qué suposiciones tiene cada lado sobre ese formato. Ignorar el protocolo es tan grave como ignorar el mecanismo.

CONCEPTO · APLICADO AL CASO

Ariane 5, leído como conector roto

Componentes	Sistema de Referencia Inercial (SRI) y sistema de control de vuelo
Tipo de conector	Llamada a procedimiento con paso de datos (valor de velocidad horizontal)
Protocolo esperado	Un valor representable en un entero de 16 bits
Lo que realmente ocurrió	Un valor de 64 bits que superó ese rango, sin manejo de excepción definido en el protocolo

El componente no cambió entre el Ariane 4 y el Ariane 5. El contexto que alimentaba el conector, sí — y nadie revisó si el protocolo seguía siendo válido.

TENSIÓN CONCEPTUAL

¿Reutilizar un componente probado es más seguro?

Postura habitual: reutilizar código ya probado en producción reduce el riesgo, comparado con escribir algo nuevo desde cero.

Lo que muestra Ariane 5: un componente "probado" solo está probado **en el contexto y el protocolo para el que fue validado**. Reutilizarlo en un contexto distinto, sin revisar esas suposiciones, puede ser más riesgoso que escribir algo nuevo con los supuestos correctos desde el inicio.

¿Cómo se decide, entonces, cuándo un componente reutilizado necesita revalidación completa?



INTERACCIÓN

Identifiquen el tipo de conector

Para cada interacción, identifiquen el tipo de conector de la taxonomía de hoy:

1. Un frontend que llama a un endpoint REST y espera la respuesta antes de continuar.
2. Un sistema de notificaciones que se activa automáticamente cuando otro componente publica un nuevo pedido, sin que el primero lo invoque directamente.
3. Varios servicios que leen y escriben sobre la misma base de datos central.
4. Un proceso que transforma un archivo y pasa el resultado, en secuencia, a otro proceso que lo comprime.

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. Un **conector** (Garlan & Shaw) representa la interacción entre componentes: mecanismo más protocolo.
2. Existen cuatro tipos principales: **llamada a procedimiento, invocación implícita, flujos de datos y acceso a datos compartidos**.
3. El **protocolo** define las reglas exactas de esa interacción — formato de datos, orden, manejo de errores.
4. Ariane 5 (1996) demuestra que un componente correcto puede causar un desastre si el conector que lo une al resto del sistema no revalida sus supuestos en un nuevo contexto.
5. Reutilizar código "probado" no elimina el riesgo si cambia el protocolo o el contexto de uso.

INSTRUCCIONES

Taller de hoy — Implementación de ejemplo

Demostración guiada por el docente — 30 a 40 minutos

El docente muestra en vivo, sobre el backend Spring Boot de sesiones anteriores, **un conector con protocolo mal definido** — igual en espíritu al fallo de Ariane 5, a escala de código— y cómo un protocolo explícito lo previene.

Un protocolo sin validar, y uno que sí valida

```
// SIN protocolo explícito: asume que el dato
// siempre cabe en el tipo esperado (como el SRI).
public class ConversorService {
    public short convertir(long valorOriginal) {
        return (short) valorOriginal; // desbordamiento silencioso
    }
}

// CON protocolo explícito: el conector define
// qué hacer si el dato no cumple el contrato esperado.
public class ConversorService {
    public short convertir(long valorOriginal) {
        if (valorOriginal > Short.MAX_VALUE || valorOriginal < Short.MIN_VALUE) {
            throw new IllegalArgumentException(
                "Valor fuera de rango para este conector: " + valorOriginal);
        }
        return (short) valorOriginal;
    }
}
```

No hay entregable de estudiantes: el objetivo es reconocer, en código real, la diferencia entre un conector con protocolo implícito y uno con protocolo explícito y validado.

CIERRE

Para la próxima sesión

- Identificar, en el proyecto propio de Fábrica Escuela, al menos un conector entre dos componentes y su protocolo.
- Repasar los lenguajes y frameworks vistos en la Semana 2 — se retoman el jueves desde la perspectiva de "stack completo".

El jueves cerramos la Unidad 1 con el stack tecnológico: cómo se combinan lenguaje, framework, base de datos e infraestructura en conjuntos con nombre propio (LAMP, MEAN, MERN, WAMP, XAMP).

